

SYNTHETIX · SECURITY REVIEW

Synthetix Treasury Market.

A security review of Treasury Market.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Synthetix	Treasury Market	Guardian	January 29 – February 11, 2025	February 18, 2025

§ 00

Contents.

61 pages — overview, confidence ranking, scope, 45 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 31 findings	P. 10
	Remediation Review · 14 findings	P. 45
§ 05	Disclaimer	P. 60
§ 06	About Guardian	P. 61

§ 01

Overview.

Guardian reviewed Treasury Market across 2 review rounds. The final report includes 5 Critical, 7 High, 13 Medium, 20 Low findings.

AUDIT FIRM	REPOSITORY
Guardian	Synthetixio/synthetix-v3
CLIENT	METHODOLOGY
Synthetix	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
Treasury Market	c6a45fc10ca9
PREPARED BY	
Guardian	
AUDIT WINDOW	
January 29 – February 11, 2025	
FINAL REPORT	
February 18, 2025	

1.1 Confidence Ranking

2 / 5

Low Confidence

Guardian assigns a Confidence Ranking of 2 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
■ Critical	5	0	3	0	2
■ High	7	0	6	0	1
■ Medium	13	0	10	1	2
■ Low	20	0	18	0	2
■ Informational	0	0	0	0	0
All severities	45	0	37	1	7

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence ASSIGNED	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Treasury Market in `Synthetixio/synthetix-v3`. The scope includes 297 source lines across 3 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
markets/treasury-market/contracts/TreasuryMarket.sol	285	434
markets/treasury-market/contracts/SynthetixTreasuryProxy.sol	8	17
markets/treasury-market/contracts/InitialModuleBundle.sol	4	12

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	New Pool Debt Distribution Not Updated	Logical Error	CRITICAL	ACKNOWLEDGED	p. 10
C-02	unsaddle Reentrancy	—	CRITICAL	RESOLVED	p. 11
C-03	saddle Is Not Enforced	—	CRITICAL	ACKNOWLEDGED	p. 12
C-04	artificialDebt Stays 0 If Debt == targetDebt	Logical Error	CRITICAL	RESOLVED	p. 13
H-01	Rebalance Sandwich Attack	—	HIGH	ACKNOWLEDGED	p. 14
H-02	Missing Rebalance Updates	Logical Error	HIGH	ACKNOWLEDGED	p. 15
H-03	Rewards Can Be Siphoned By Migrations	Logical Error	HIGH	RESOLVED	p. 17
H-04	Liquidation Issues	Logical Error	HIGH	ACKNOWLEDGED	p. 18
H-05	Parallel Market Liquidation Risk	—	HIGH	ACKNOWLEDGED	p. 20
M-01	Pool collateral limit can be bypassed	—	MEDIUM	RESOLVED	p. 21
M-02	Smart Contract Accounts Cannot Unsaddle	—	MEDIUM	RESOLVED	p. 22
M-03	Race Condition in Adjust Loan and Unsaddle	—	MEDIUM	ACKNOWLEDGED	p. 23
M-04	Missing Vault C-ratio Validation	—	MEDIUM	PARTIALLY RESOLVED	p. 24
M-05	Early Return In Rebalance May DoS The Market	—	MEDIUM	ACKNOWLEDGED	p. 25

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-06	Unsaddle May Be Blocked For Last User	Logical Error	MEDIUM	ACKNOWLEDGED	p. 26
M-07	Exit DoS If accountDebt > artificialDebt	—	MEDIUM	ACKNOWLEDGED	p. 27
L-01	Unnecessary Sender Computation	—	LOW	RESOLVED	p. 29
L-02	Unnecessary Current Loan Calculation	—	LOW	RESOLVED	p. 30
L-03	Delegates Cannot Perform Actions	—	LOW	ACKNOWLEDGED	p. 31
L-04	Unsaddle Prevented By Min Delegation Time	—	LOW	ACKNOWLEDGED	p. 32
L-05	Treasury Market Out Of Range Risk	—	LOW	ACKNOWLEDGED	p. 33
L-06	Wrong comment	—	LOW	ACKNOWLEDGED	p. 34
L-07	Missing Event On Critical Param Change	—	LOW	ACKNOWLEDGED	p. 35
L-08	Loan duration is not validated	—	LOW	ACKNOWLEDGED	p. 36
L-09	Incomplete comment	—	LOW	ACKNOWLEDGED	p. 37
L-10	Missing Interface Support	—	LOW	ACKNOWLEDGED	p. 38
L-11	Reported Debt Does Not Validate marketId	—	LOW	ACKNOWLEDGED	p. 39
L-12	New Stakers Can Join Pool	Logical Error	LOW	ACKNOWLEDGED	p. 40
L-13	Treasury Market Lacks Upside	—	LOW	ACKNOWLEDGED	p. 41
L-14	Vault Liquidation via Unsaddle and Max Withdrawable	Logical Error	LOW	ACKNOWLEDGED	p. 42
L-15	Unsaddle Race Condition	Logical Error	LOW	ACKNOWLEDGED	p. 44

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
REMEDATION REVIEW					
C-01	Low Cratio Accounts Immediately Gain	—	CRITICAL	ACKNOWLEDGED	p. 45
H-01	burnUsd Perturbs Locks	Logical Error	HIGH	ACKNOWLEDGED	p. 46
H-02	Incentive For Bypassing UI-Enforced Rebalance	—	HIGH	ACKNOWLEDGED	p. 47
M-01	Early Return In Rebalance May DoS The Market	Logical Error	MEDIUM	ACKNOWLEDGED	p. 48
M-02	Users Can Saddle Below 200% C-Ratio, Shifting Debt to Treasury	—	MEDIUM	ACKNOWLEDGED	p. 49
M-03	Potential Systemic Risks of Disabling Liquidations	Logical Error	MEDIUM	ACKNOWLEDGED	p. 51
M-04	The Migrated From Vault Could Become Unhealthy	—	MEDIUM	ACKNOWLEDGED	p. 52
M-06	Missing Check In mintTreasury	—	MEDIUM	ACKNOWLEDGED	p. 53
M-07	Unsaddle May Be Blocked For Last User	Logical Error	MEDIUM	ACKNOWLEDGED	p. 54
L-01	Missing requireSufficientDelegation Validation	Logical Error	LOW	ACKNOWLEDGED	p. 55
L-02	Invalid Loan Completion Percentage	Logical Error	LOW	ACKNOWLEDGED	p. 56
L-03	Insufficient Check In _repaymentPenalty	—	LOW	ACKNOWLEDGED	p. 57
L-04	setDebtDecayFunction May Be Called Too Late	Logical Error	LOW	ACKNOWLEDGED	p. 58
L-05	Treasury Debt Management Warning	—	LOW	ACKNOWLEDGED	p. 59

CRITICAL **C-01**

ACKNOWLEDGED

New Pool Debt Distribution Not Updated

CATEGORY	Logical Error
LOCATION	VaultModule.sol
REVIEW	Main Review

DESCRIPTION

In the `migrateDelegation` function there is no `updateAccountDebt` invocation for the new pool, therefore the pending debt distribution is not made before the account is assigned a `valuePerShare` for the account debt distribution.

This means that the account will experience whatever pending debt distribution there is from before the migration is made.

RECOMMENDATION

Update the account debt distribution for the new pool before assigning the `valuePerShare` to the account with the `assignDebtToAccount` function.

RESOLUTION

Status: Acknowledged.

CRITICAL **C-02**

RESOLVED

unsaddle Reentrancy

CATEGORY —
LOCATION TreasuryMarket.sol
REVIEW Main Review

DESCRIPTION

`TreasuryMarket.unsaddle()` repays the debt of the given user by withdrawing USD from the system, which increased the `netIssuanceD18` of the market and in turn its `totalDebt` as well. That's why `_rebalance()` is called after `unsaddle` completes, to account for the change in `netIssuanceD18` by adjusting the `artificialDebt`.

However, before calling `_rebalance()`, the account `NFT` is returned back to the user by `safeTransferFrom()`. This will invoke the receiver's `onERC721Received` and will allow them to call functions like `saddle` on the treasury market or `migrateDelegation` to take advantage of the stale state.

For example, a user may re-enter into the V3 core system by calling `migrateDelegation` and stand to immediately benefit by the `reportedDebt` change that will occur directly after their migration via the `_rebalance` call.

RECOMMENDATION

Execute `_rebalance()` before the ERC721 transfer.

RESOLUTION

Status: Resolved.

CRITICAL C-03

ACKNOWLEDGED

saddle Is Not Enforced

CATEGORY	—
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The Synthetix frontend will call `saddle` on the TreasuryMarket contract whenever a user delegates to its pool, but this function call is not enforced on-chain.

When one or more users delegate collateral to the pool without calling `saddle` the first user who calls `saddle` will receive a lot more debt than the user should, as the `targetDebt` of the first user calling `saddle` is calculated based on the total collateral in the vault:

```
targetDebt = vaultCollateralValue.divDecimal(targetCratio).toInt();
```

This either leads to the user not being able to `saddle` as the `associateDebt` call reverts because this makes the user liquidatable, or the `artificialDebt` is inflated and the user acquires way more debt than the user should.

Furthermore, this issue allows users to delegate a minimal amount and saddle it as debt before subsequently delegating a larger amount and allowing it to accumulate debt before saddling this amount as well and having this debt paid off by the treasury market without an additional loan.

RECOMMENDATION

Always call `saddle` when a user delegates to the pool, in the same transaction.

RESOLUTION

Status: Acknowledged.

CRITICAL C-04

RESOLVED

artificialDebt Stays 0 If Debt == targetDebt

CATEGORY Logical Error
LOCATION TreasuryMarket.sol: 186
REVIEW Main Review

DESCRIPTION

When a user enters the system (delegates to the pool and calls `saddle`) with a CR equal to the `targetCratio`, the `artificialDebt` will stay 0 as the calculated `targetDebt` will equal the account's debt, and the `artificialDebt` is increased like that:

```
artificialDebt += targetDebt - accountDebt;
```

The first user who calls `saddle` after that will receive a lot more debt than the user should, as the `targetDebt` is calculated based on the total collateral in the vault, if the `artificialDebt` is 0 (because the system thinks this is the first user who enters the system):

```
targetDebt = vaultCollateralValue.divDecimal(targetCratio).toInt();
```

This either leads to the user not being able to `saddle` as the `associateDebt` call reverts because this makes the user liquidatable, or the `artificialDebt` is inflated and the user acquires way more debt than the user should.

RECOMMENDATION

Use a different method to check if this is the first user entering the system.

RESOLUTION

Status: Resolved.

HIGH **H-01**

ACKNOWLEDGED

Rebalance Sandwich Attack

CATEGORY —
LOCATION TreasuryMarket.sol
REVIEW Main Review

DESCRIPTION

The `rebalance` function updates the Treasury market's artificial debt to match the target collateralization ratio. This will often be done when sister markets attached to the same pool accrue debt such that the collateralization ratio is smaller than desired.

However because the rebalance is an immediate stepwise change in the `reportedDebt` of the treasury market, malicious actors can arbitrage the update and extract immediate value from a reduction in reported debt in the rebalance function.

This arbitrage can take place by migrating to the new pool, triggering the rebalance function to reduce the `reportedDebt` of the treasury market and thus realizing a net debt reduction on your position, and then subsequently unsaddling with this reduced debt.

RECOMMENDATION

Consider making the rebalancing a smooth linear function that adjusts in real time in the `reportedDebt` function rather than stepwise jumps which can be potentially arbitrated.

RESOLUTION

Status: Acknowledged.

HIGH **H-02**

ACKNOWLEDGED

Missing Rebalance Updates

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianAudits/treasury-market-2/blob/c6a45fc10ca9168524d8bc655b31521a8df5c9d5/markets/treasury-market/contracts/TreasuryMarket.sol#L175 , https://github.com/GuardianAudits/treasury-market-2/blob/c6a45fc10ca9168524d8bc655b31521a8df5c9d5/markets/treasury-market/contracts/TreasuryMarket.sol#L231
REVIEW	Main Review

DESCRIPTION

A rebalance should be enforced before calling `delegate` and `saddle`, as without it, the vault debt value would return a stale artificial debt.

For example, if the price of the collateral token increases or decreases after the last rebalance and the artificial debt is not updated, the vault debt value considered in the following code line would be incorrect:

```
[TreasuryMarket.sol#L175](https://github.com/GuardianAudits/treasury-market-2/blob/c6a45fc10ca9168524d8bc655b31521a8df5c9d5/markets/treasury-market/contracts/TreasuryMarket.sol#L175)
```

It is important to note that the vault debt value depends on artificial debt via `reportedDebt`.

Failing to update this value would result in either inflated or deflated debt being assigned to accounts, impacting both `associateDebt` and `loanAmount` calculations.

For `unsaddle`, the impact is not persisted as it is for `saddle` due to a rebalance occurring at the end. However, stale `reportedDebt` still affects the `accountDebt` being repaid.

```
[TreasuryMarket.sol#L231](https://github.com/GuardianAudits/treasury-market-2/blob/c6a45fc10ca9168524d8bc655b31521a8df5c9d5/markets/treasury-market/contracts/TreasuryMarket.sol#L231)
```

This repayment happens by withdrawing USD.

Consider a case where the actual artificial debt is lower than what is stored in the contract state. In such a case, the `accountDebt` repaid would be higher than the original amount. Depending on available liquidity, this may unnecessarily cause a revert when it should not.

Ultimately, this does not pose a long-term issue, as the increase in artificial debt due to rebalance in the end would compensate for the additional amount being repaid.

H-02 Missing Rebalance Updates

CONTINUED

The `setTargetCRatio` function is used to update the `targetCRatio` of the `TreasuryMarket` and calls `_rebalance` at the end.

The `_rebalance` function will reset the `artificialDebt` to zero if the `artificialDebt` is smaller or equal to the `vaultDebtValue` minus the calculated `targetDebt`.

This is possible with a `targetCRatio` update depending on the given `targetCRatio` and the current state of the market and would lead to major consequences.

The next user who calls `saddle` will receive a lot more debt than the user should, as the `targetDebt` of the first user calling `saddle` is calculated based on the total collateral in the vault:

```
targetDebt = vaultCollateralValue.divDecimal(targetCratio).toInt();
```

This either leads to the user not being able to `saddle` as the `associateDebt` call reverts because this makes the user liquidatable, or the `artificialDebt` is inflated and the user acquires way more debt than the user should.

— RECOMMENDATION

Consider enforcing a rebalance before both delegation + saddle, unsaddle and `setTargetCRatio` operations.

Note: A rebalance cannot be performed immediately before saddle. Instead, it must be executed before the combination of delegation and saddle to ensure accurate debt calculations.

— RESOLUTION

Status: Acknowledged.

HIGH **H-03**

RESOLVED

Rewards Can Be Siphoned By Migrations

CATEGORY Logical Error
LOCATION VaultModule.sol: 160
REVIEW Main Review

DESCRIPTION

In the `migrateDelegation` function there is no `updateRewardsToVaults` function call for the new pool. As a result the pending rewards are not streamed to exclusively the prior existing delegators but now instead the pending reward distribution is now credited to the migrated delegate as well.

RECOMMENDATION

Call `updateRewardsToVaults` for the new pool in addition to the old pool.

RESOLUTION

Status: Resolved.

HIGH **H-04**

ACKNOWLEDGED

Liquidation Issues

CATEGORY	Logical Error
LOCATION	TreasuryMarket.sol
REVIEW	Main Review

DESCRIPTION

When users get liquidated in the core system, their collateral and debt is reset to zero and are distributed across the other participants in the given vault. This behavior causes problems in the `TreasuryMarket`.

When users `saddle` and are liquidated before the `unsaddle` call, their `saddledCollateral` value will stay as it is. The `unsaddle` action can not go through after this because `delegateCollateral` will revert since it sets the user collateral to 0 and it is already zero since the user has been liquidated. The next time the user saddles, `newlySaddledValue` will be heavily miscalculated. Either the transaction will revert because of subtraction underflow or it will go through using a wrong `newlySaddledValue` (if the new saddled value is greater than what's recorded in `saddledCollateral`).

Because `saddledCollateral` is not 0, `newlySaddledDebt` will be 0, even though the user might have migrated a large amount of debt. This perturbs the vault ratio and the `artificialDebt` calculation. Furthermore, because `saddledCollateral` is not 0, the user's debt won't be created as a loan as well, so they won't have to pay it.

The liquidation will increase the collateral and debt balance of the other participants as well. If one of these users calls `saddle` while their `saddledCollateral` is not zero, the ratio will be perturbed again and their debt won't be added to their loan.

Furthermore liquidated users are still able to pay off their treasury market loan with the `adjustLoan` function and may unknowingly be burning this payment amount as their account has already been liquidated.

Individual and vault liquidations are a risk in the event that either too much value is extracted with the `treasuryMint` function or if connecting markets experience a significant enough debt increase over time, which cannot be covered by the yield of the treasury market.

H-04 Liquidation Issues

CONTINUED

— RECOMMENDATION

Firstly, consider adding validation in the `mintTreasury` function such that it is not possible to purposefully or accidentally withdraw sUSD to the point where it places the backing delegates or vault at increased risk of liquidation. E.g. do not allow the `artificialDebt` to be lowered below a specific threshold, since if it is lowered too far then the Treasury market will have insufficient ability to offset future pool debt increases.

Secondly, consider preventing users who have been liquidated from paying off their treasury market loan by validating that the collateral amount in their account is nonzero.

Finally, ideally liquidations can be handled appropriately by the Treasury market. In order to do so the delegation/migration and saddling logic can check two specific states to determine if the account was previously liquidated and account for it accordingly.

The collateral of the account in the v3 system before the current delegation/migration can be validated against 0, if the previous delegated collateral amount is 0 then the saddled collateral can be reset to 0 as the account was liquidated. Finally if the vault epoch has increased relative to the last recorded vault epoch in saddle for this account then the account saddled collateral can be reset.

— RESOLUTION

Status: Acknowledged.

HIGH **H-05**

ACKNOWLEDGED

Parallel Market Liquidation Risk

CATEGORY —

LOCATION TreasuryMarket.sol L104, L365

REVIEW Main Review

DESCRIPTION

The Treasury market uses artificial debt to compensate for what the target debt should be according to the target collateral ratio, and reports this as `reportedDebt` to the pool.

This artificial debt is calculated by considering total collateral value and total debt of the vault, including parallel markets.

According to our communication with the SNX team, the pool for Treasury market is designed to have other parallel markets. However, these parallel markets neither consider artificial debt nor trigger rebalancing of the Treasury market when their debt positions change.

Consequently, when a new debt position is taken according to the allowed weight on any parallel market, that debt is double-counted in artificial debt, and therefore in reported debt, causing total vault debt to be inflated.

This inflation increases the debt-collateral ratio for the vault and raises liquidation risk until the Treasury market is rebalanced.

An adversary can exploit this flaw by causing parallel markets to report a high debt for a short period, and potentially triggering a vault liquidation or individual delegation liquidations before the protocol or other parties have a chance to rebalance.

RECOMMENDATION

Consider making the `reportedDebt` function a smooth function which does not depend on a rebalancing to occur in the treasury market.

PROOF OF CONCEPT <https://github.com/GuardianAudits/treasury-market-2/pull/5/files>

RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

RESOLVED

Pool collateral limit can be bypassed

CATEGORY —

LOCATION VaultModule.sol

REVIEW Main Review

DESCRIPTION

When users delegate collateral to a pool, `VaultModule` ensures the new delegation is not over the maximum allowed value for that pool by calling `checkPoolCollateralLimit()`.

This restriction can be bypassed with the new `migrateDelegation()` functionality. Users can migrate their collateral from a pool with higher limit to another with a lower one and put the second pool over its allowed limit.

Because the `checkPoolCollateralLimit` will revert if the token to be delegated is not supported by the pool, this makes it possible for users to delegate a collateral token from pool X to another pool Y which doesn't support that token.

RECOMMENDATION

Add a `checkPoolCollateralLimit()` call for the new pool in `migrateDelegation()`

PROOF OF CONCEPT <https://github.com/GuardianAudits/treasury-market-1/blob/migrate-unsupported-collateral-POC/markets/treasury-market/test/TreasuryMarket.test.sol>

RESOLUTION

Status: Resolved.

MEDIUM **M-02**

RESOLVED

Smart Contract Accounts Cannot Unsaddle

CATEGORY —

LOCATION TreasuryMarket.sol: 265

REVIEW Main Review

DESCRIPTION

The `unsaddle` function uses ERC721's `safeTransferFrom` function to transfer an account token back to the sender address. However if the sender address is a Smart Contract without an `onERC721Received` function then this transfer reverts and the unsaddle cannot take place.

RECOMMENDATION

Consider using the normal `transferFrom` from the ERC721 interface so that even contracts which do not have an `onERC721Received` implementation may unsaddle. The fact that this sending address owns an account NFT is evidence enough that the contract can sufficiently handle an account NFT.

RESOLUTION

Status: Resolved.

MEDIUM **M-03**

ACKNOWLEDGED

Race Condition in Adjust Loan and Unsaddle

CATEGORY —

LOCATION TreasuryMarket.sol L296 L211

REVIEW Main Review

— DESCRIPTION

The purpose of `adjustLoan` is to allow users to repay their initial debt with a penalty and then `unsaddle` to exit the market.

However, since these are separate actions, another user could execute `unsaddle` in between, withdrawing the liquidity created by first user's `adjustLoan`, leaving first user waiting for a new saddle.

— RECOMMENDATION

Consider making `adjustLoan` and `unsaddle` an atomic action by providing helper function.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

PARTIALLY RESOLVED

Missing Vault C-ratio Validation

CATEGORY —

LOCATION VaultModule.sol: 160

REVIEW Main Review

DESCRIPTION

In the `migrateDelegation` function there is no validation that the new vault c-ratio is not in a liquidatable state during the migration. If this is the case then the migration will result in a position that is immediately liquidated along with the vault.

Furthermore, there is no validation that the old vault c-ratio is valid at the time of migration. This means users may migrate out of a liquidatable vault to avoid the vault liquidation.

RECOMMENDATION

Include the `_verifyPoolCratio` validation in the `migrateDelegation` function for both the old vault and the new vault similarly to the `delegateCollateral` function.

RESOLUTION

Status: Partially resolved.

MEDIUM **M-05**

ACKNOWLEDGED

Early Return In Rebalance May DoS The Market

CATEGORY —

LOCATION TreasuryMarket.sol L413

REVIEW Main Review

DESCRIPTION

When the treasury mints new snxUSD, artificial debt decreases because actual debt increases on the V3 system side. Consider a scenario where the treasury mints to full capacity, setting `artificial_debt = 0`.

If the treasury later repays some debt using `burnTreasury`, this reduces actual debt on the V3 system. This should increase `artificial_debt` to reflect the target debt.

However, this doesn't happen because rebalancing returns early when artificial debt is zero.

```
TreasuryMarket.sol L413
function _rebalance() internal {
if (artificialDebt == 0) { return; }
...
}
```

As a result, once artificial debt reaches zero, if no new saddle is done, since `burnTreasury` can't restore the artificial debt. It blocks both `unsaddle` (since `unsaddle` requires `artificialDebt > 0` due to `artificialDebt -= repayment`) and further `mintTreasury` calls.

RECOMMENDATION

Consider removing the early return `artificialDebt == 0` case in the `_rebalance` function.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

ACKNOWLEDGED

Unsaddle May Be Blocked For Last User

CATEGORY Logical Error
LOCATION TreasuryMarket.sol: 240
REVIEW Main Review

— DESCRIPTION

During the unsaddle function, a check is performed which prevents the last user from exiting if they are the only remaining LP in the vault, as their collateral constitutes the entire vault balance.

— RECOMMENDATION

The check could be modified to:

```
if (accountCollateral > vaultCollateral) {  
    revert ParameterError.InvalidParameter(  
        "accountCollateral",  
        "no surplus collateral to fund exit"  
    );  
}
```

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

ACKNOWLEDGED

Exit DoS If accountDebt > artificialDebt

CATEGORY —

LOCATION TreasuryMarket.sol: 247-249

REVIEW Main Review

DESCRIPTION

When users call `saddle` the `artificialDebt` is incremented by the difference between the calculated `targetDebt` and the account's debt.

Between `saddle` and `unsaddle` the debt of users can increase when the debt of the legacy market rises. When users call `unsaddle` the `artificialDebt` is decremented by the debt of the account

Therefore depending on the user's CR at `saddle` time and the accrued debt from the legacy market between `saddle` and `unsaddle` the debt of all users which can unsaddle can be bigger than the `artificialDebt`.

The `unsaddle` function is used to bring the account's debt to 0 and undelegate the account from the pool. To accomplish this the market will:

- withdraw `snxUSD`
- deposit the funds into the user's account
- burn the `snxUSD` in the user's account to decrease the account's debt
- undelegate the account from the pool (only possible with zero debt)

But as the withdrawal of `snxUSD` will increase the total debt of the treasury market and therefore the user's debt it is necessary to either decrease the total debt of the market or burn even more `snxUSD` to bring the user's debt to 0. The protocol decided to decrease the total debt of the market by decreasing the `artificialDebt` before calling `withdrawMarketUsd`.

But as mentioned above the account's debt can be bigger than the `artificialDebt` and as the `reportedDebt` is capped at 0 it is not possible to decrease the debt of the market below 0. The `unsaddle` function is therefore not able to bring the user's debt to 0 if the user's debt is bigger than the `artificialDebt`. This will lead to a DoS and frozen user funds as the attempt to undelegate the account will fail if there is still debt left in the account.

M-07 Exit DoS If accountDebt > artificialDebt

CONTINUED

RECOMMENDATION

If the `artificialDebt` decrement hits zero, withdraw and deposit excess funds to bring the account's debt to 0.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

Unnecessary Sender Computation

CATEGORY —

LOCATION TreasuryMarket.sol: 219

REVIEW Main Review

DESCRIPTION

In the `unsaddle` function the `accountToken` owner validation re-computes the sender address in the `Unauthorized` revert data when instead the same stored `sender` address may be used.

RECOMMENDATION

Consider using the same `sender` stack variable in the `Unauthorized` revert data instead of re-computing the sender.

RESOLUTION

Status: Resolved.

LOW **L-02**

RESOLVED

Unnecessary Current Loan Calculation

CATEGORY —

LOCATION TreasuryMarket.sol: 224

REVIEW Main Review

DESCRIPTION

In the `unsaddle` function the `currentLoan` value is computed twice. Once on line 222 to compute the `currentLoan` value and validate that it is 0. And a second time on line 224 in the `OutstandingLoan` revert data to compute the `outstandingLoanAmount` value.

RECOMMENDATION

Instead of recomputing the `currentLoan` value, re-use the `currentLoan` variable in the `OutstandingLoan` revert data.

RESOLUTION

Status: Resolved.

LOW **L-03**

ACKNOWLEDGED

Delegates Cannot Perform Actions

CATEGORY —

LOCATION TreasuryMarket.sol: 211, 298

REVIEW Main Review

DESCRIPTION

In the `unsaddle` and `adjustLoan` functions there is validation to ensure that only the owner of an account token is allowed to unsaddle. However in the core V3 system there are delegation roles which allow third parties to manage the collateral and other features of token accounts.

The lack of delegation access to the `unsaddle` function may limit access for third party systems to the Treasury market.

RECOMMENDATION

Consider allowing delegates for an account with a certain role to unsaddle or adjust the loan for the account owner.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Unsaddle Prevented By Min Delegation Time

CATEGORY —

LOCATION TreasuryMarket.sol

REVIEW Main Review

— DESCRIPTION

The act of unsaddling includes an undelegation from the backing pool, however this may unexpectedly revert for users when their unsaddling action is completed before the end of the minimum delegation period for an attached market in the backing pool.

— RECOMMENDATION

This may be the expected behavior and necessary to ensure sufficient delegation time, however it should be documented for users and integrators of the treasury market.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Treasury Market Out Of Range Risk

CATEGORY —
LOCATION TreasuryMarket.sol
REVIEW Main Review

DESCRIPTION

In some exceptional edge cases it may be possible for the Treasury market to be bumped out of range if the debt per credit capacity limit is hit. This will remove the treasury market's ability to control the pool's debt and can lead to a DoS for users attempting to unsaddle their positions.

RECOMMENDATION

Adding validations on the lowest the artificial debt is able to be assigned during the `mintTreasury` function and perhaps even the `unsaddle` function can sufficiently address the risk of this edge case ever arising.

RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Wrong comment

CATEGORY —
LOCATION VaultModule.sol:189
REVIEW Main Review

— DESCRIPTION

The following [comment](https://github.com/GuardianAudits/treasury-market-1/blob/c6a45fc10ca9168524d8bc655b31521a8df5c9d5/protocol/synthetix/contracts/modules/core/VaultModule.sol#L189) says `Cannot migrate if c-ratio is in liquidation`, but the next line doesn't check the c-ratio, it updates the debt distribution chain.

— RECOMMENDATION

Update the comment.

— RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

Missing Event On Critical Param Change

CATEGORY —

LOCATION TreasuryMarket.sol: 344-363

REVIEW Main Review

DESCRIPTION

There are two functions that allow the owner to change critical parameters of the treasury market:

`setTargetCratio` and `setDebtDecayFunction`. The `setTargetCratio` function emits an event, but the `setDebtDecayFunction` does not. Best practice is to emit an event when critical system parameters are updated.

RECOMMENDATION

Consider adding an event for the `setDebtDecayFunction` function.

RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

Loan duration is not validated

CATEGORY —
LOCATION TreasuryMarket.sol
REVIEW Main Review

DESCRIPTION

There are validations inside `setDebtDecayFunction` for each parameter except `time`. This means the `duration` of the loan can be set to an unreasonable value.

RECOMMENDATION

Validate the `time` parameter as well.

RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

Incomplete comment

CATEGORY —
LOCATION TreasuryMarket.sol
REVIEW Main Review

— DESCRIPTION

The comment inside `TreasuryMarket.minimumCredit()` says `we lock collateral here because` and doesn't explain why.

— RECOMMENDATION

Complete the comment by explaining that users should not be able to undelegate from the pool and also migrate from it.

— RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

Missing Interface Support

CATEGORY —

LOCATION TreasuryMarket.sol 381-390

REVIEW Main Review

DESCRIPTION

The `supportsInterface` function does not return that the `IERC721Receiver` interface is supported.

RECOMMENDATION

Consider adding the `IERC721Receiver` interface to the `supportsInterface` function.

RESOLUTION

Status: Acknowledged.

LOW **L-11**

ACKNOWLEDGED

Reported Debt Does Not Validate marketId

CATEGORY —

LOCATION TreasuryMarket.sol: 104

REVIEW Main Review

DESCRIPTION

In the function `reportedDebt`, the argument `requestedMarketId` is never validated. This differs from other markets, where `requestedMarketId` is checked against the current market and returns `0` if there is no match.

Without this validation, integration errors could occur, potentially leading to unintended behavior.

RECOMMENDATION

Validate `requestedMarketId` to match `marketId`.

RESOLUTION

Status: Acknowledged.

LOW **L-12**

ACKNOWLEDGED

New Stakers Can Join Pool

CATEGORY	Logical Error
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

According to SIP-420, only existing SNX stakers should be allowed to join the new protocol-owned debt pool.

However, in the current implementation, new stakers can bypass this restriction by using `delegateCollateral`. This could contradict the intended design of the Treasury Market and should be addressed.

— RECOMMENDATION

To strictly abide by SIP-420, consider enabling only `migrateDelegation`.

— RESOLUTION

Status: Acknowledged.

LOW **L-13**

ACKNOWLEDGED

Treasury Market Lacks Upside

CATEGORY —

LOCATION TreasuryMarket.sol: 211

REVIEW Main Review

— DESCRIPTION

In the `unsaddle` function the Treasury market only pays the user's outstanding debt if it is positive, but does not take the user's accumulated profit when the accumulated debt is negative.

This means that the Treasury market does not hold exposure to the profits in excess of the original debt assigned to the account. This should not be the case because it presents an unfairly beneficial offer to the user at the detriment of the protocol.

Users can rest assured that any additional debt accumulated over the saddled period will be covered by the treasury market, but still have exposure to the upside from profits generated by markets attached to the pool.

This is an increasingly large issue because users are not required to unsaddle as soon as their loan has expired, they can remain saddled for an extended period of time to reap the benefits of having their downsides covered while still holding exposure to the upside.

— RECOMMENDATION

Mint any negative debt value to the treasury address to take the upside of the user's position for the treasury market.

— RESOLUTION

Status: Acknowledged.

LOW **L-14**

ACKNOWLEDGED

Vault Liquidation via Unsaddle and Max Withdrawable

CATEGORY Logical Error
LOCATION TreasuryMarket.sol
REVIEW Main Review

DESCRIPTION

In unsaddle, the treasury market enables users to exit their positions by subtracting neededToPay from artificialDebt and withdrawing USD from the pool:

```
artificialDebt -= int256(neededToRepay);  
v3System.withdrawMarketUsd(marketId, address(this), neededToRepay);
```

One would assume that `artificialDebt > neededToRepay` is always enforced, as `withdrawMarketUsd` should revert otherwise. However, as demonstrated in the Proof of Concept, `getWithdrawableMarketUsd` allows users to withdraw funds up to the collateral value instead.

Exploit Scenario

If `neededToRepay > artificialDebt`, the user can withdraw more than `**artificialDebt**` to close their saddle position, exposing the entire vault to liquidation risk.

Example (From POC):

1. User A has 4 ETH of collateral delegated and 2 ETH of debt.
2. They perform saddle.
3. Since the debt capacity is fully utilized, `artificialDebt` becomes zero, meaning the vault is already at maximum utilization.
4. Ideally, further withdrawals should be blocked.
5. However, the PoC demonstrates that an additional 2 ETH can still be withdrawn. (PoC uses `mintTreasury` to demonstrate `withdrawMarketUsd` for simplicity, but the same effect could be achieved by `unsaddle` as well. `rebalance` won't revert either, since it would reset `artificialDebt` to 0)
6. This results in a total vault debt of 4 ETH, making it vulnerable to liquidation.

POC:

```
function test_liq_vault() external {  
    sideMarket.setReportedDebt(2 ether);
```

L-14 Vault Liquidation via Unsaddle and Max Withdrawable

CONTINUED

```
sideMarket.callAssociateDebt(poolId, address(collateralToken), accountId, 2 ether);
market.saddle(accountId);
console.log('artificialDebt', market.reportedDebt(0));
console.log('withdrawable: ', v3System.getWithdrawableMarketUsd(market.marketId()));
vm.prank(market.owner());
market.mintTreasury(2 ether);
console.log(v3System.isVaultLiquidatable(poolId, address(collateralToken)));
}
```

— RECOMMENDATION

Consider enforcing `artificialDebt >= neededToPay` in `unsaddle`. Furthermore, it may be prudent to add a similar validation in the `mintTreasury` function to avoid treasury minting from putting the vault or individual positions at liquidation risk.

— RESOLUTION

Status: Acknowledged.

LOW **L-15**

ACKNOWLEDGED

Unsaddle Race Condition

CATEGORY Logical Error
LOCATION Global
REVIEW Main Review

— DESCRIPTION

The treasury market tries to earn yield on Ethena and repay the debt of all users with it which delegated to the market for a given amount of time.

If for any reason the market is not able to earn enough funds to do that the users who `unsaddle` first will get all of their debt forgiven and the others will be stuck in the system.

— RECOMMENDATION

Consider adding a fair mechanism so that all users can exit under the same conditions in such an emergency situation. For example everyone gets only a portion of their debt forgiven and they are able to pay the rest themselves. Instead of a few users getting all of their debt forgiven and the others being stuck.

— RESOLUTION

Status: Acknowledged.

CRITICAL **C-01**

ACKNOWLEDGED

Low Cratio Accounts Immediately Gain

CATEGORY	—
LOCATION	Global
REVIEW	Remediation Review

— DESCRIPTION

Because delegation/migration and saddling is not atomically enforced to occur in the same transaction a malicious actor can go around the UI enforcements and steal value from the rebalancing system.

The malicious actor can specifically put their account in a C-ratio below the target C-ratio of the treasury market before calling the `migrateDelegation` function. Then upon having their position migrated to the new pool, the actor can trigger a rebalance on the Treasury market which will correct for their lower C-ratio and immediately forgive a portion of their debt.

This attack can be repeated with large volume many times to drain the pool of nearly all USD value and ultimately liquidate the backing vault.

— RECOMMENDATION

Require that the `delegateCollateral` and `migrateCollateral` functions call into the Treasury market saddle function to automatically saddle the user in the same transaction at the contract level.

— RESOLUTION

Status: Acknowledged.

HIGH **H-01**

ACKNOWLEDGED

burnUsd Perturbs Locks

CATEGORY	Logical Error
LOCATION	IssueUSDModule.sol: 115
REVIEW	Remediation Review

— DESCRIPTION

In the `burnUsd` function there is no validation that the amount of sUSD collateral being burned to repay debt is not locked.

This allows the account to enter an invalid state of having locked collateral larger than their current collateral balance. And furthermore allows the account to bypass the lock mechanism and extract value from their locked collateral by immediately paying down debt with it.

A user can trivially deposit sUSD collateral, lock it, and whenever they wish to withdraw it before the lock they can `mintUsd` to accrue debt and burn the locked sUSD to repay the debt.

— RECOMMENDATION

Validate that the burned collateral is not locked in the `burnUsd` function.

— RESOLUTION

Status: Acknowledged.

HIGH **H-02**

ACKNOWLEDGED

Incentive For Bypassing UI-Enforced Rebalance

CATEGORY —
LOCATION TreasuryMarket.sol L201
REVIEW Remediation Review

DESCRIPTION

To address the previously reported H2 issue, the Synthetix team proposed triggering rebalance from the UI for each interaction instead of enforcing it via the contract.

However, this approach introduces following problem:

If the collateral value drops and an account's debt exceeds the actual target debt, the following check should ideally prevent saddling:

```
if (accountDebt > targetDebt) {  
    revert InsufficientCRatio(accountId, accountDebt.toUint(), targetDebt.toUint());  
}
```

However, if rebalance is not executed before calling delegate and saddle, the `targetDebt` remains inflated, allowing users to saddle even when their actual C-ratio is too low.

Ultimately, the treasury is forced to cover the shortfall, creating an unintended liability.

These accounts benefit from partial debt repayment in the next rebalance, effectively shifting the burden to the treasury.

Since rebalance is not enforced on-chain, users can manually call delegate and saddle, bypassing the UI-triggered rebalance.

RECOMMENDATION

To prevent this exploit, rebalance should be enforced at the contract level for any combination of `delegate/migrate` and `saddle`.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Early Return In Rebalance May DoS The Market

CATEGORY Logical Error
LOCATION TreasuryMarket.sol L413
REVIEW Remediation Review

DESCRIPTION

When the treasury mints new snxUSD, artificial debt decreases because actual debt increases on the V3 system side. Consider a scenario where the treasury mints to full capacity, setting `artificial_debt = 0`.

If the treasury later repays some debt using `burnTreasury`, this reduces actual debt on the V3 system. This should increase `artificial_debt` to reflect the target debt.

However, this doesn't happen because rebalancing returns early when artificial debt is zero.

```
TreasuryMarket.sol L413
function _rebalance() internal {
if (artificialDebt == 0) { return; }
...
}
```

As a result, once artificial debt reaches zero, if no new saddle is done, since `burnTreasury` can't restore the artificial debt. It blocks both `unsaddle` (since `unsaddle` requires `artificialDebt > 0` due to `artificialDebt -= repayment`) and further `mintTreasury` calls.

RECOMMENDATION

Consider removing the early return `artificialDebt == 0` case in the `_rebalance` function.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Users Can Saddle Below 200% C-Ratio, Shifting Debt to Treasury

CATEGORY —

LOCATION TreasuryMarket.sol L201

REVIEW Remediation Review

DESCRIPTION

In `**saddle()**`, users are allowed to saddle as long as their account debt remains $\leq$ target debt.

```
function saddle(uint128 accountId) external override {  
    /.....  
    targetDebt = vaultCollateralValue.divDecimal(targetCratio).toInt();  
    } else if (vaultCollateralValue > accountCollateralValue) {  
        .....  
        targetDebt =  
        (accountCollateralValue.toInt() * (vaultDebtValue - newlySaddledDebt)) /  
        (vaultCollateralValue - newlySaddledValue).toInt();  
    }  
    if (accountDebt > targetDebt) {  
        revert InsufficientCRatio(accountId, accountDebt.toUint(), targetDebt.toUint());  
    }  
    .....  
}
```

The target debt is intended to maintain a target collateralization ratio (C-ratio) of 200% relative to the vault's collateral. Currently, the target debt calculation is done proportionally to the vault's existing debt.

This approach works *as long as the calculated * `targetDebt <= vaultCollateralValue.divDecimal(targetCratio).toInt()`

Once `targetDebt > vaultCollateralValue.divDecimal(targetCratio).toInt()` becomes issue as demonstrated in following case,—users would be able to saddle even if their individual account debt exceeds the 200% collateralization ratio.

Once a user saddles under such conditions, the burden of repaying the excess debt shifts to the treasury.

M-02 Users Can Saddle Below 200% C-Ratio, Shifting Debt to Treasury

CONTINUED

Example Scenario:

1. Initial State

- Collateral Value: 4 ETH
- Debt: 2 ETH
- Artificial Debt: 0 ETH
- C-ratio: 200% (4 ETH / 2 ETH)
- Treasury Minted Debt: 2 ETH

2. Collateral Price Drops by 10%

- Updated Collateral Value: 3.6 ETH
- Debt: 2 ETH
- Artificial Debt: 0 ETH
- New C-ratio: 180% (3.6 ETH / 2 ETH)

3. Issue with Saddle Mechanism

- At this point, the user should only be able to saddle if their C-ratio $\geq 200\%$
- However, since the system recalculates the target debt based on the adjusted vault debt, it still allows saddling even when the user's individual C-ratio falls below 200%.
- The user, now having 3.6 ETH collateral value with 2 ETH debt, can still saddle, leading to a treasury liability.

— RECOMMENDATION

Even if you:

- Rebalance before delegating + saddle,
- Use atomic delegation + saddle or migrate delegation + saddle,

the issue still persists.

Consider comparing account debt to `min (targetDebt, accountCollateralValue.divDecimal(targetCratio).toInt())`

It is also advised to the Treasury that they should never mint the total available capacity and should keep some buffer for artificial debt to absorb price updates like this.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Potential Systemic Risks of Disabling Liquidations

CATEGORY Logical Error
LOCATION Global
REVIEW Remediation Review

— DESCRIPTION

To address the previously reported H-04/H-05 issues, the Synthetix team has proposed completely disabling liquidations for the associated pool and allowing unsaddling on a first-come, first-served (FCFS) basis.

However, we would like to highlight the potential impact of this decision—not just on the Treasury Market, but on the entire SNX ecosystem if such a scenario unfolds.

In V3 Core, the sUSD used within Treasury Market is fundamentally no different from the sUSD used in other pools.

Therefore, if Treasury Market's pool becomes insolvent—meaning the sUSD minted by the Treasury is no longer fully backed—it could have system-wide consequences for SNX.

If the reasoning behind this decision is that the Treasury controls the minting and burning of sUSD for the associated pool, and would therefore hold any unbacked balance, this assumption is not entirely accurate.

Whenever the Treasury swaps sUSD for Ethena USD to earn additional yield, the sUSD minted within Treasury Market enters the open market.

If insolvency occurs, a significant amount of unbacked sUSD could lead to a bank run, where users rush to withdraw collateral from other pools, or swapping it to other stables, potentially causing an sUSD depeg.

— RECOMMENDATION

Reconsider the decision to completely disable liquidations for the Treasury Market's associated pool, as it introduces systemic risk to the SNX ecosystem.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

ACKNOWLEDGED

The Migrated From Vault Could Become Unhealthy

CATEGORY —

LOCATION VaultModule.sol: 245

REVIEW Remediation Review

— DESCRIPTION

The `_verifyPoolCratio` validation is not performed on the old pool in the `migrateDelegation` function. As a result a user may make the from vault liquidatable by removing their delegation from that vault.

— RECOMMENDATION

Perform the `_verifyPoolCratio` for the old pool in the `migrateDelegation` function.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

ACKNOWLEDGED

Missing Check In mintTreasury

CATEGORY —
LOCATION TreasuryMarket.sol: 363-367
REVIEW Remediation Review

DESCRIPTION

The protocol only partially fixed the `L-14 Vault Liquidation via Unsaddle and Max Withdrawable` finding and left the `mintTreasury` function unchanged. This report describes how the likelihood of the issue in the `mintTreasury` function can be increased.

The owner of the protocol could accidentally mint the vault into a liquidatable state in the following scenario:

- The protocol owner creates a transaction to mint a lot of sUSD (but not so much that the vault will be liquidated)
- Malicious actor front runs and unsaddles a large amount
- The transaction of the protocol goes through and the vault is left in a liquidatable state

RECOMMENDATION

Ensure in the `mintTreasury` function that the minted amount will not push the vault into a liquidatable state.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

ACKNOWLEDGED

Unsaddle May Be Blocked For Last User

CATEGORY Logical Error
LOCATION TreasuryMarket.sol: 240
REVIEW Remediation Review

DESCRIPTION

During the unsaddle function, a check is performed which prevents the last user from exiting if they are the only remaining LP in the vault, as their collateral constitutes the entire vault balance.

RECOMMENDATION

The check could be modified to:

```
if (accountCollateral > vaultCollateral) {  
    revert ParameterError.InvalidParameter(  
        "accountCollateral",  
        "no surplus collateral to fund exit"  
    );  
}
```

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Missing requireSufficientDelegation Validation

CATEGORY	Logical Error
LOCATION	Missing `requireSufficientDelegation` in the migration function
REVIEW	Remediation Review

DESCRIPTION

As of now, `migrateDelegation` does not enforce a minimum delegation amount, unlike `delegateCollateral`.

If the intention is to keep `minDelegationAmount` as 0, as specified in the test configuration, then this is not an issue. However, if there are plans to introduce such a limit in the future, consider addressing this.

RECOMMENDATION

Consider adding check of `requireSufficientDelegation` inside `migrateDelegation`.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Invalid Loan Completion Percentage

CATEGORY	Logical Error
LOCATION	TreasuryMarket.sol: 467
REVIEW	Remediation Review

DESCRIPTION

In the `_repaymentPenalty` function the loan completion percentage is assigned to 0 in the event that the loan duration is 0.

However in the event that the loan duration is 0 then the loan would be instantly entirely completed.

RECOMMENDATION

Assign the `loanCompletionPercentage` to 1e18 in the event that the loan duration is 0.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Insufficient Check In `_repaymentPenalty`

CATEGORY —
LOCATION TreasuryMarket.sol: 463-465
REVIEW Remediation Review

DESCRIPTION

The `_repaymentPenalty` function early returns zero if the target loan is greater than the current loan as no changes are made in that case:

```
if (targetLoan > currentLoan || currentPenalty == 0) { return 0; }
```

A `>=` makes more sense here as nothing changes in this case either.

RECOMMENDATION

Change the mentioned line to `if (targetLoan > currentLoan || currentPenalty == 0) { return 0; }.`

RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

setDebtDecayFunction May Be Called Too Late

CATEGORY	Logical Error
LOCATION	TreasuryMarket.sol
REVIEW	Remediation Review

— DESCRIPTION

The market starts when the `registerMarket` function is called which already sets a default `targetCratio` but all the debt decaying params are not set to default values here.

Therefore if the market is deployed and registered and a user enters into the market before the debt decaying params are set, the user gets a 0 loan power and 0 loan duration.

These are very bad conditions for the user the loan will not decay and the user has to pay the maximum penalty when he wants to exit.

— RECOMMENDATION

Consider setting default debt decaying params in the `registerMarket` function. To ensure that everyone enters the market with fair conditions.

— RESOLUTION

Status: Acknowledged.

LOW L-05

ACKNOWLEDGED

Treasury Debt Management Warning

CATEGORY	—
LOCATION	Global
REVIEW	Remediation Review

— DESCRIPTION

The treasury has a responsibility to carefully manage the artificial debt of the treasury market.

It is important that the treasury does not mint the entire available amount from the treasury market so that a reasonable amount of artificial debt is left behind to absorb price movements and side market activities.

— RECOMMENDATION

Be aware that the treasury market must carefully balance the artificial debt in this way.

— RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.